



**Universitat  
de Lleida**



ESCOLA  
POLITÀCNICA SUPERIOR  
UNIVERSITAT DE LLEIDA

---

# What's GraphQL

Activity 2: Distributed computing applications report

June 5, 2026

May Castells Raga  
Anna Marin Nuño

Distributed Computing  
Grau en Enginyeria Informàtica  
Universitat de Lleida

---

# Contents

## 1 First Problem (3.5 Points) – Division of a negative integer

We consider the program in ‘division.c’ that computes the integer division of a negative number  $D$  by a positive number  $d$ . We restricted the inputs using  $_{C}PROVER_{assume} to D \in [-600, -1]$  and  $d \in [1, 600]$  to avoid unbounded loop unwinding and arithmetic overflows.

Key results (see ‘informe/division\_c\_bmc.txt’ for full output):

CBMC was run with ‘-unwind 601 -unwinding-assertions -bounds-check’.

The check for the division invariant ‘ $D == q*d + r$ ’ failed in our run (see log), indicating either an algorithmic bug or a missing pre/post-condition.

Arithmetic overflow checks were exercised; bounding ‘d’ reduced overflow cases.

## 2 Second Problem (3 Points) – Maximum even in array

The program ‘simpleWhile.c’ finds the maximum even number in an array of size ‘N’. We added the following assertions to validate correctness:

- If ‘maxeven != 1’ then ‘maxeven’ is even and equals the greatest even element.
- If ‘maxeven == 1’ then every element in the array is odd.

CBMC was executed with ‘-unwind 11 -bounds-check’ and the resulting log is in ‘informe/simpleWhile\_checks\_and\_assertions\_passed\_for\_the\_chosen\_unwinding’.

## 3 Third Problem (3.5 Points)

Consider the following C program that contains a sort function, that it is not implemented, and it should sort an array of N integers. It also contains a function, checkSort, that it is intended to be used for testing, with assertions, that the sort function works as expected. Per trobar la K, es necessita aquestes dues proves:

- Amb  $K = 4$  iteracions encara existeix una execució on el bucle es podria desenredar un cop més.

```
cbmc -verbosity 8 -no-standard-checks -trace -unwind 4 -unwinding-assertions
sort.c
```

```
VERIFICATION FAILED, Violated property:  file sort.c function checkSort
line 33 thread 0 unwinding assertion loop 0
```

- Amb  $K = 5$  iteracions ja no existeix cap iteració on el bucle es desenredi aquest nombre de cops.

```
cbmc -verbosity 8 -no-standard-checks -trace -unwind 5 -unwinding-assertions
sort.c
```

1. Implement the sort function with your favourite sorting algorithm.

```

1 void sort (int8_t a[], int size) {
2     for (int i = 0; i < size - 1; i++)
3     {
4
5         for (int j = 0; j < size - i - 1; j++)
6         {
7             if (a[j] > a[j + 1])
8                 swap(a, j, j + 1);
9         }
10    }
11 }
12
13
14 void swap(int8_t *arr, int i, int j)
15 {
16     int8_t temp = arr[i];
17     arr[i] = arr[j];
18     arr[j] = temp;
19 }
20

```

2. Write assertions at the end of the checkSort function to verify if your sorting algorithm works as expected. Check with cbmc the function checkSort, but with five different values of N: 4, 6, 8, 10, 12. Write down the computation time needed by cbmc to verify the program with these four different array sizes. Do you think that the computation time increases exponentially ?

| N    | 4      | 6     | 8     | 10    | 12    |
|------|--------|-------|-------|-------|-------|
| time | 0.100s | 0.146 | 0.282 | 0.694 | 1.933 |

Table 1: Caption

3. Introduce an small error in the implementation of your sort function. Then, check it again (with the minimum size for N) to see if cbmc finds the error.